

Entwicklung einer Observability-Strategie für die Plattform "frag.jetzt" basierend auf dem Konzept der Four Golden Signals.

Exposé zur Bachelorarbeit im Studiengang Informatik

Leon Rausch

20. Januar 2026

1 Einleitung

Webbasierte Anwendungssysteme werden heute häufig als verteilte, cloud-native Anwendungen umgesetzt, die aus mehreren lose gekoppelten Services bestehen. Diese Architekturform erlaubt eine unabhängige Weiterentwicklung einzelner Komponenten sowie eine flexible Skalierung, erhöht jedoch zugleich den Aufwand für Betrieb, Überwachung und Fehleranalyse im Produktivsystem [1, 2]. Insbesondere bei Microservices entstehen verteilte Fehlerzustände und komplexe Abhängigkeiten zwischen Diensten, wodurch Ursachen von Störungen nicht mehr eindeutig einzelnen Komponenten zugeordnet werden können [3].

Vor diesem Hintergrund gewinnt Observability als Erweiterung klassischer Monitoring-Konzepte an Bedeutung. Ziel von Observability ist es, den internen Zustand eines Systems anhand extern erfassbarer Anzeichen nachvollziehen zu können. Dazu werden Metriken, Logs und Traces kombiniert, um Zusammenhänge zwischen Systemverhalten, Fehlern und Leistungsmerkmalen sichtbar zu machen [4, 1]. Empirische Untersuchungen aus dem Bereich cloud-nativer Anwendungen zeigen, dass ein strukturierter Einsatz von Observability-Praktiken die Analyse von Incidents unterstützt und die Reaktionszeit im Betrieb verkürzen kann [5, 2].

Ein häufig verwendeter Orientierungsrahmen zur Strukturierung servicebezogener Beobachtungsdaten sind die von Google im Kontext des Site Reliability Engineering (SRE) beschriebenen *Four Golden Signals*: Latenz, Traffic, Fehler und Sättigung [6]. Diese Signalarten erfassen grundlegende technische Symptome eines Dienstes und eignen sich als Ausgangspunkt zur Ableitung erster Monitoring-Indikatoren. Für den produktiven Betrieb liefern sie jedoch primär Rohinformationen, die erst durch Kontext, Interpretation und Bewertung für Menschen handlungsrelevant werden. Google SRE weist selbst darauf hin, dass die Wirksamkeit solcher Signale maßgeblich davon abhängt, wie sie in Alerting, Entscheidungsprozesse und betriebliche Abläufe eingebettet sind [6].

Die Open-Source-Plattform *frag.jetzt* verfügt über eine solide Entwicklungs- und Deployment-Infrastruktur,

jedoch ist die Erfassung und Auswertung von Betriebsdaten derzeit nicht als durchgängiges Observability-Konzept ausgelegt. Metriken, Logs, Traces und Alerting werden nicht konsistent entlang des gesamten Anfragepfads betrachtet, wodurch Zusammenhänge zwischen Nutzerverhalten, Backend-Verarbeitung und externen Abhängigkeiten nur eingeschränkt nachvollziehbar sind. Diese Einschätzung stützt sich auf die öffentlich verfügbare Projektdokumentation und Architekturübersicht des *frag.jetzt*-Repositories [7] sowie auf allgemeine Beobachtungen zu typischen Defiziten in vergleichbaren Systemen [5].

Ziel dieser Arbeit ist die Entwicklung einer Observability-Strategie für *frag.jetzt*, bei der technische Messgrößen systematisch mit nutzungsbezogenen Fragestellungen verknüpft werden. Die Four Golden Signals dienen dabei als unterstützender Referenzrahmen zur Strukturierung der Beobachtungsdaten, nicht jedoch als alleinige Grundlage für betriebliche Entscheidungen. Die Signale werden servicespezifisch für das PWA-Frontend, das Spring-Boot-Backend, die PostgreSQL-Datenbank sowie angebundene KI-Dienste konkretisiert und in messbare Indikatoren überführt. Ergänzend wird untersucht, welche Open-Source-Observability-Werkzeuge die Anforderungen von *frag.jetzt* im Hinblick auf Erfassung und betriebliche Nutzbarkeit am besten erfüllen.

Ein weiterer Schwerpunkt der Arbeit liegt auf der Ausgestaltung eines Alerting-Ansatzes, der Auffälligkeiten innerhalb der Plattform mit Fragen der Relevanz und Dringlichkeit verknüpft. Dabei steht nicht allein die Detektion von Abweichungen im Vordergrund, sondern insbesondere die Bewertung, ob eine Handlung erforderlich ist, wen ein Problem betrifft und welche Auswirkungen es auf die Nutzung der Plattform hat. Forschungsarbeiten zeigen, dass eine Kombination aus priorisiertem Alerting und datenbasierten Analyseverfahren dazu beitragen kann, irrelevante Alarmierungen zu reduzieren und menschliche Entscheidungsprozesse im Betrieb zu unterstützen [8]. Aufbauend auf den definierten Alerts werden Runbooks konzipiert, die strukturierte Diagnose- und Handlungsschritte für wiederkehrende Incident-Szenarien bereitstellen.

Die Forschungsfragen dieser Arbeit lauten daher: (1) Wie können die Four Golden Signals für die spezifischen Services von *frag.jetzt* (PWA-Frontend, Spring Boot Backend, PostgreSQL, KI-Services) konkret definiert und instrumentiert werden? (2) Welche Observability-Stack-Kombination (Prometheus/Grafana, ELK Stack, Jaeger) ist am besten geeignet für die Monitoring-Anforderungen von *frag.jetzt*? (3) Wie kann ein intelligentes Alert-System entwickelt werden, das False Positives minimiert und actionable insights für verschiedene Stakeholder-Rollen (DevOps, Entwickler, Product Owner) bereitstellt?

2 Theoretische Fundierung: Monitoring und Observability in modernen Systemen

Moderne Softwarelösungen sind zunehmend durch verteilte Architekturen, Cloud-native Technologien und eine hohe Änderungsdynamik geprägt. Insbesondere Microservice-basierte Anwendungen mit Container-Orchestrierung und Continuous Deployment erhöhen die Komplexität des Betriebs erheblich [2]. Klassische Monitoring-Ansätze, die primär auf vordefinierte Metriken und statische Schwellwerte setzen, stoßen in solchen Umgebungen an ihre Grenzen, da sie häufig nur bekannte Fehlerzustände erfassen und wenig Unterstützung bei der Ursachenanalyse bieten [4].

Vor diesem Hintergrund hat sich der Ansatz der *Observability* als wichtiges Konzept für den Betrieb moderner Systeme etabliert. Observability zielt darauf ab, den internen Zustand eines Systems anhand externer Messwerte nachvollziehbar zu machen und dadurch auch unbekannte oder unerwartete Fehlerszenarien analysierbar zu gestalten [9].

2.1 Begriffliche Grundlagen: Monitoring und Observability

Der Begriff Observability stammt ursprünglich aus der System- und Regelungstheorie und beschreibt die Fähigkeit, interne Zustände eines Systems aus dessen Ausgaben abzuleiten. Übertragen auf heutige Softwareanwendungen bezeichnet Observability die Eigenschaft, das Verhalten einer Application durch geeignete Telemetriedaten nachvollziehbar zu machen [4].

Monitoring und Observability sind dabei eng miteinander verbunden, jedoch nicht gleichzusetzen. Monitoring fokussiert sich auf die Überwachung bekannter Metriken und Ereignisse, um Abweichungen vom erwarteten Normalzustand zu erkennen. Observability erweitert diesen Ansatz, indem sie Kontextinformationen bereitstellt, die eine detaillierte Analyse von Ursachen und Zusammenhängen ermöglichen [1]. In der Literatur wird Observability daher häufig als Weiterentwicklung oder Übermenge des klassischen Monitorings beschrieben.

Zentral für Observability ist die systematische Erfassung unterschiedlicher Telemetriearten. Erst durch die kombinierte Betrachtung verschiedener Daten wird es möglich, komplexe Systemzustände sowie Ursachen von Fehlern zuverlässig zu analysieren [9, 2].

2.2 Kernbausteine der Observability: Metriken, Logs, Traces und Alerting

In der Literatur werden vier Hauptelemente der Observability unterschieden: Metriken, Logs, Traces und Alerting. Diese Bausteine ergänzen sich gegenseitig und bilden gemeinsam die Grundlage für eine umfassende Analyse des Laufzeitverhaltens moderner Softwaresysteme [9, 2].

Metriken sind numerische Messwerte, die den Zustand oder das Verhalten eines einer Anwendung über die Zeit hinweg beschreiben, beispielsweise Antwortzeiten, Fehlerraten oder Ressourcenauslastung. Sie eignen sich besonders für Trendanalysen, Kapazitätsplanung und die Definition von Service-Level-Zielen. Aufgrund ihrer aggregierten Form ermöglichen Metriken eine effiziente Überwachung auch großer Softwaresysteme [2].

Logs bestehen aus ereignisbasierten Textinformationen, die detaillierte Einblicke in den Ablauf einzelner Systemkomponenten liefern. Sie enthalten kontextreiche Informationen, etwa Fehlermeldungen oder Zustandsänderungen, und sind insbesondere für die Ursachenanalyse und das Debugging grundlegend. Der Nachteil von Logs liegt in ihrem hohen Volumen und der vergleichsweise aufwändigen Auswertung [9].

Traces dienen der Nachverfolgung einzelner Anfragen über mehrere Services hinweg. Sie zeigen, welche Komponenten an der Verarbeitung beteiligt waren und wie viel Zeit in den jeweiligen Verarbeitungsschritten verbracht wurde. Distributed Tracing ist insbesondere in Microservice-Architekturen ein essenzielles Mittel zur Identifikation von Latenzengpässen und Abhängigkeitsproblemen [10].

Alerting ergänzt die passive Beobachtung durch aktive Benachrichtigung bei kritischen Zuständen. Alerts werden typischerweise auf Basis von Metriken oder Log-Ereignissen ausgelöst und sollen Betreiber frühzeitig auf Probleme hinweisen. Eine zentrale Herausforderung besteht darin, Alarmregeln so zu gestalten, dass relevante Vorfälle erkannt werden, ohne eine hohe Anzahl an Fehlalarmen zu erzeugen [8].

Erst das Zusammenspiel dieser vier Bausteine ermöglicht eine umfassende Observability. Während Metriken einen schnellen Überblick liefern, stellen Logs und Traces den notwendigen Detailgrad für die Ursachenanalyse bereit. Alerting sorgt schließlich dafür, dass Abweichungen zeitnah erkannt und adressiert werden können.

2.3 Die Four Golden Signals nach Google SRE

Die *Four Golden Signals* stellen ein technisches Referenzmodell zur Beobachtung von Diensten dar und wurden im Rahmen des Site Reliability Engineering von Google entwickelt [6]. Sie beschreiben vier wesentliche technische Messgrößen, anhand derer sich Symptome von Leistungs- oder Stabilitätsproblemen identifizieren lassen:

- **Latency:** Die Zeitspanne zur Bearbeitung einer Anfrage. Hohe oder stark schwankende Latenzen weisen auf Performanceprobleme hin.
- **Traffic:** Das Anfrage- oder Datenvolumen, das auf einen Dienst einwirkt, beispielsweise gemessen in Requests pro Sekunde.
- **Error:** Die Rate fehlgeschlagener Anfragen, etwa durch Serverfehler oder funktionale Fehlzustände.
- **Saturation:** Der Auslastungsgrad kritischer Ressourcen wie CPU, Speicher oder Warteschlangen.

Google empfiehlt, diese Signale als erste Orientierung im Monitoring zu nutzen, betont jedoch zugleich, dass ihre Aussagekraft stark vom jeweiligen Nutzungskontext abhängt [6]. Insbesondere für Alerting und operative Entscheidungen ist eine Ergänzung durch Kontextinformationen erforderlich, etwa zur aktuellen Nutzungssituation, zur betroffenen Nutzergruppe oder zur zeitlichen Relevanz eines Problems. Ohne diese Einbettung besteht die Gefahr, dass technisch korrekte Anzeichen zu unnötigen oder nicht handlungsrelevanten Alarmierungen führen.

2.4 Die Plattform *frag.jetzt* und ihre Systemarchitektur

Die vorliegende Arbeit bezieht sich exemplarisch auf die Q&A-Plattform *frag.jetzt*, eine Cloud-native Webanwendung mit serviceorientierter Architektur. Die Plattform wurde an der Technischen Hochschule Mittelhessen entwickelt und dient der Durchführung interaktiver Fragerunden. Neben klassischer Frage-Antwort-Funktionalität integriert *frag.jetzt* auch KI-gestützte Assistenzfunktionen zur automatisierten Beantwortung von Nutzeranfragen [7].

Zur Einordnung der späteren Observability-Strategie wird im Folgenden ein Überblick über die Architekturkomponenten der Plattform gegeben [7]:

- **Angular Frontend (SPA/PWA):** Eine Single-Page-Application, über die Nutzer mit dem System interagieren. Das Frontend ist als Progressive Web App umgesetzt und kommuniziert überwiegend über HTTPS-APIs sowie WebSockets mit dem Backend.
- **WebSocket-Gateway (NGINX):** Ein Gateway-Server zur Vermittlung der bidirektionalen Echtzeitkommunikation zwischen Frontend und Backend, insbesondere für das asynchrone Pushen von Antworten und Statusupdates.

- **Spring Boot Backend:** Der zentrale, Java-basierte Anwendungsserver, der die Kernlogik der Plattform implementiert. Er verarbeitet Nutzeranfragen, koordiniert interne Dienste und stellt REST- sowie WebSocket-Schnittstellen bereit.
- **Keycloak Identity Service:** Ein externer Dienst zur Authentifizierung und Autorisierung von Nutzern. Er übernimmt das Identitätsmanagement, führt jedoch zusätzliche Abhängigkeiten und potenzielle Latenzpunkte ein.
- **PostgreSQL-Datenbank:** Eine relationale Datenbank zur Persistierung des Anwendungszustands, etwa für Benutzerkonten, Fragen und Antworten. Aufgrund ihrer Rolle bei der Verarbeitung und Speicherung aller zustandsbehafteten Daten stellt sie einen kritischen Ausfallpunkt, einen sogenannten Single Point of Failure, bei hoher Last dar.
- **KI-Backend (Langchain & FastAPI):** Ein eigenständiger Microservice, der über Python (FastAPI) und die Langchain-Bibliothek Anfragen an externe Large-Language-Modelle weiterleitet, um KI-generierte Antworten zu erzeugen. Die Abhängigkeit von externen KI-APIs bringt variable Antwortzeiten und zusätzliche Fehlerquellen mit sich.
- **RabbitMQ Message Broker:** Ein Message-Broker zur asynchronen Entkopplung von Backend- und KI-Komponenten. RabbitMQ ermöglicht das Puffern und Weiterleiten von Antwortnachrichten und unterstützt so eine entkoppelte und skalierbare Verarbeitung.

Diese verteilten Komponenten bilden gemeinsam den durchgängigen Anfragepfad der Plattform. Eine Nutzerfrage durchläuft typischerweise mehrere Dienste, bevor eine Antwort an das Frontend zurückgegeben wird. Dadurch entstehen zahlreiche Schnittstellen und potenzielle Fehlerquellen. Eine Observability-Strategie für *frag.jetzt* muss daher sicherstellen, dass entlang dieses gesamten Pfades geeignete Metriken, Logs und Traces erfasst werden, um Performanceprobleme und Ausfälle nachvollziehbar analysieren zu können [1].

2.5 Werkzeuge für Monitoring und Observability

Zur Umsetzung von Monitoring- und Observability-Konzepten steht eine Vielzahl spezialisierter Open-Source-Werkzeuge zur Verfügung, die in modernen Cloud- und Microservice-Architekturen etabliert sind. Diese Werkzeuge adressieren unterschiedliche Aspekte der Telemetrieerfassung und -auswertung und werden häufig in Kombination eingesetzt [11].

- **Prometheus:** Eine Open-Source-Software zur Erfassung und Speicherung zeitbasierter Metriken. Prometheus arbeitet mit einem Pull-basierten Ansatz, bei dem Metrik-Endpunkte regelmäßig abgefragt werden. Die integrierte Abfragesprache PromQL erlaubt detaillierte Analysen von Zeitreihendaten und bildet die Grundlage für regelbasierte

Alerting-Mechanismen. Prometheus ist als CNCF-Projekt weit verbreitet und insbesondere für dynamische Cloud-Umgebungen geeignet [11].

- **Grafana:** Eine Plattform zur Visualisierung und Analyse von Observability-Daten aus unterschiedlichen Datenquellen. Grafana dient häufig als Dashboard-Werkzeug, in dem Metriken, Logs und Traces gemeinsam dargestellt und korreliert werden können. Dadurch unterstützt Grafana die visuelle Analyse von Systemzuständen und Trends [5].
- **Elastic Stack (ELK):** Der Elastic Stack kombiniert Elasticsearch, Logstash und Kibana und wird vor allem für die zentrale Aggregation und Analyse von Logdaten eingesetzt. Die Stärke des Stacks liegt in der skalierbaren Speicherung unstrukturierter Daten sowie in leistungsfähigen Such- und Analysefunktionen. Neben Logs unterstützt Elastic auch Metriken und Tracing, erfordert jedoch vergleichsweise hohe Systemressourcen [11].
- **Jaeger:** Ein verteiltes Tracing-Tool zur Nachverfolgung von Anfragen über Service-Grenzen hinweg. Jaeger ermöglicht die Analyse von End-to-End-Latenzen und unterstützt die Identifikation von Performanceengpässen sowie Abhängigkeitsbeziehungen in Microservice-Architekturen [10].
- **OpenTelemetry:** Ein herstellerneutraler Standard zur Instrumentierung von Anwendungen. OpenTelemetry (OTel) stellt ein einheitliches API und SDK zur Erfassung von Metriken, Logs und Traces bereit und erlaubt die Weiterleitung dieser Daten an unterschiedliche Backends. Dadurch wird die Telemetrieerfassung von der Wahl konkreter Observability-Werkzeuge entkoppelt [10].

Die Auswahl geeigneter Werkzeuge erfolgt typischerweise anhand mehrerer Bewertungskriterien, darunter die unterstützten Telemetriarten, Integrationsfähigkeit, Skalierbarkeit, Analyse- und Visualisierungsfunktionen sowie der Reifegrad der jeweiligen Lösung [5]. Diese Kriterien bilden die Grundlage für eine spätere vergleichende Analyse im Rahmen der Methodik.

3 Methodik / Forschungsdesign

Die Arbeit folgt einem artefaktorientierten Forschungsdesign mit konzeptionellem und prototypischem Schwerpunkt. Ziel ist es, eine praxisnahe Observability-Strategie für *frag.jetzt* zu entwickeln und diese anhand konkreter Artefakte nachvollziehbar darzustellen. Die Methodik gliedert sich in sechs aufeinander aufbauende Schritte.

3.1 Schritt 1: Service-spezifische Konkretisierung der Four Golden Signals

Im ersten Schritt werden die Four Golden Signals als Referenzmodell herangezogen und für die Zielarchitektur von *frag.jetzt* konkretisiert. Für jedes Teilsystem

(PWA-Frontend, Spring-Boot-Backend, PostgreSQL-Datenbank und KI-Dienste) werden geeignete Interpretationen für die einzelnen Beobachtungsgrößen abgeleitet, beispielsweise Latenz aus Nutzersicht, Fehlerraten pro Service oder Ressourcenauslastung kritischer Komponenten. Ziel ist die Definition klar interpretierbarer Indikatoren, die sowohl für Dashboards als auch für Alerting nutzbar sind.

3.2 Schritt 2: Vergleichende Evaluation von Observability-Stack-Kombinationen

Der zweite Schritt umfasst eine vergleichende Evaluation geeigneter Observability-Werkzeuge. Da eine vollständige praktische Erprobung aller verfügbaren Tools im Rahmen der Arbeit nicht realistisch ist, erfolgt der Vergleich primär auf theoretischer und konzeptioneller Ebene. Untersucht werden Prometheus/Grafana, der Elastic Stack sowie Jaeger unter Berücksichtigung der Anforderungen von *frag.jetzt*. Bewertungskriterien sind unter anderem die Abdeckung der benötigten Telemetriarten, die Integrationsfähigkeit in bestehende Systeme, der Betriebsaufwand sowie die Verfügbarkeit als Open-Source-Lösung. Auf dieser Basis wird eine begründete Auswahl eines geeigneten Stacks getroffen.

3.3 Schritt 3: Instrumentierung mit OpenTelemetry

Im dritten Schritt wird die Instrumentierung der Services konzipiert. OpenTelemetry dient dabei als einheitliche Instrumentierungswerkzeug zur Erfassung von Metriken, Logs und Traces. Ziel ist eine konsistente Telemetrie entlang des gesamten Anfragepfads, die eine Korrelation zwischen Nutzerinteraktionen, Backend-Verarbeitung und externen Abhängigkeiten ermöglicht.

3.4 Schritt 4: Rollenbasierte Dashboards

Der vierte Schritt befasst sich mit der Konzeption rollenbasierter Dashboards. Für unterschiedliche Zielgruppen wie DevOps, Entwickler und Product Owner werden Sichten entworfen, die Telemetriedaten mit nutzungs- und betriebsrelevantem Kontext verknüpfen. Ziel ist es, nicht nur Auffälligkeiten darzustellen, sondern deren Bedeutung für die jeweilige Rolle nachvollziehbar zu machen, etwa im Hinblick auf Nutzerwahrnehmung, Dringlichkeit oder notwendige Maßnahmen.

3.5 Schritt 5: Alerting-Framework mit Fokus auf False-Positive-Reduktion

Im fünften Schritt wird ein Alerting-Ansatz entwickelt, der sich an den Golden Signals orientiert und auf Prometheus als technischer Basis aufsetzt. Neben klassischen regelbasierten Alerts werden Konzepte zur Priorisierung und Reduktion irrelevanter Alarme

berücksichtigt. Dazu gehören beispielsweise die Aggregation mehrerer Signale, zeitliche Glättung sowie der Einsatz datengetriebener Verfahren wie einfacher Anomalieerkennung. Optional wird untersucht, inwiefern KI-gestützte Verfahren zur Mustererkennung oder Alert-Bewertung eingesetzt werden können, um die Anzahl unnötiger Benachrichtigungen zu reduzieren.

3.6 Schritt 6: Entwicklung operativer Runbooks

Im sechsten Schritt werden exemplarische Runbooks für wiederkehrende Incident-Szenarien erstellt. Diese Runbooks sollen Alerts mit klaren Diagnose- und Handlungsanweisungen verknüpfen und dienen dazu, Reaktionszeiten zu verkürzen und die operative Qualität der Störungsbearbeitung zu erhöhen.

3.7 Ergebnisartefakte

Aus den beschriebenen Schritten ergeben sich die geplanten Ergebnisse: (1) ein wissenschaftlicher Bericht zur Strategie, Toolauswahl und Alerting-Konzeption, (2) eine lauffähige Observability-Plattform (z. B. Prometheus/Grafana), (3) rollenbasierte Dashboards sowie (4) beispielhafte Runbooks für Incident-Situationen.

4 Geplante Gliederung (konzeptionell)

1. Einleitung

Problemstellung, Motivation, Zielsetzung und Forschungsfragen; Einordnung der Arbeit in den Kontext cloud-nativer Systeme und moderner Observability.

2. Theoretische Fundierung

Grundlagen zu Monitoring und Observability (Metriken, Logs, Traces, Alerting); Four Golden Signals als Referenzmodell; Überblick relevanter Observability-Tools und Toolklassen.

3. Untersuchungsgegenstand: Architektur von *frag.jetzt*

Beschreibung der Systemarchitektur (PWA-Frontend, Spring Boot Backend, PostgreSQL, KI-Services sowie Schnittstellen/Kommunikationspfade); Ableitung der Observability-Anforderungen je Komponente.

4. Methodik / Forschungsdesign

Vorgehensmodell der Arbeit; Operationalisierung der Golden Signals; Kriterienkatalog für Tool-/Stack-Evaluation; Instrumentierungs- und Evaluationsplan; Validitäts- und Abgrenzungsaspekte.

5. Evaluation von Observability-Stacks

Vergleich Prometheus/Grafana, ELK Stack, Jaeger (und OpenTelemetry als Instrumentierungsstandard); Ergebnisdarstellung anhand definierter Kriterien; begründete Auswahl einer geeigneten Stack-Kombination.

6. Konzeption der Observability-Strategie für *frag.jetzt*

Service-spezifische Golden-Signal-Definitionen; Messkonzept (Metriken/Logs/Traces) entlang des Anfragepfads; Telemetrie- und Korrelationserfordernisse.

7. Umsetzung: Instrumentierung und Observability-Plattform

Umsetzung der Telemetrie-Erfassung; Konfiguration der Plattformkomponenten; Nachweis der End-to-End-Sicht (Metrik-Trace-Log-Korrelation).

8. Dashboards und Alerting-Konzept

Rollenbasierte Dashboards; Alerting-Regeln auf Grundlage nutzerrelevanter Beobachtungsgrößen; Ansatz zur False-Positive-Reduktion und Priorisierung; Einbindung von Anomalieerkennung (konzeptionell/prototypisch).

9. Runbooks und Incident-Playbooks

Erstellung exemplarischer Runbooks; Verknüpfung von Alerts mit Diagnose- und Handlungsschritten; typische Incident-Szenarien.

10. Diskussion

Einordnung der Ergebnisse, Grenzen der Umsetzung, Trade-offs (z. B. Aufwand vs. Nutzen, Datenvolumen/Kardinalität, Abhängigkeit externer KI-Services).

11. Fazit und Ausblick

Zusammenfassung der wichtigsten Ergebnisse; Ausblick auf Weiterentwicklung (z. B. Reifegradsteigerung, Service Level Objective (SLO)-basierte Steuerung, weitere Automatisierung).

Abkürzungsverzeichnis

API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment
CPU	Central Processing Unit
ELK	Elasticsearch, Logstash, Kibana
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
KI	Künstliche Intelligenz
NLP	Natural Language Processing
OTel	OpenTelemetry
PWA	Progressive Web App
Q&A	Question and Answer
REST	Representational State Transfer
SLO	Service Level Objective
SPA	Single-Page Application
SRE	Site Reliability Engineering
UI	User Interface

Quellen und weiterführende Literatur

- [1] U. Faseeha u. a. „Observability in Microservices: An In-Depth Exploration of Frameworks, Challenges, and Deployment Paradigms“. In: *IEEE Access* 13 (2025), S. 72011–72035. DOI: 10.1109/ACCESS.2025.3562125.
- [2] Kowshik Sakinala. „Monitoring and Observability for Cloud-Native Applications“. In: *Journal of Computer Science and Technology Studies* 7.8 (2025), S. 101–115. URL: <https://al-kindipublishers.org/index.php/jcsts/article/view/10459>.
- [3] S. Fischer u. a. „An Overview of Microservice-Based Systems Used for Evaluation in Testing and Monitoring: A Systematic Mapping Study“. In: (2024). DOI: 10.1145/3644032.3644445.
- [4] Cindy Sridharan. *Distributed Systems Observability*. O'Reilly Media, 2018. URL: <https://unlimited.humio.com/rs/756-LMY-106/images/Distributed-Systems-Observability-eBook.pdf>.
- [5] Luca Giamattei u. a. „Monitoring tools for DevOps and microservices: A systematic grey literature review“. In: *Journal of Systems and Software* 208 (2024), S. 111906. DOI: 10.1016/j.jss.2023.111906.
- [6] Rob Ewaschuk. *Monitoring Distributed Systems*. <https://sre.google/sre-book/monitoring-distributed-systems/>. Kapitel 6 in *Site Reliability Engineering*, herausgegeben von Betssy Beyer, Google. 2017.
- [7] TH Mittelhessen und arsnova. *frag.jetzt – Open-Source Q&A-Plattform*. 2025. URL: <https://gitlab.arsnova.eu/arsnova/frag.jetzt/-/blob/staging/README.md> (besucht am 20.01.2026).
- [8] Fatemeh Jalalvand u. a. „Alert prioritisation in security operations centres: A systematic survey on criteria and methods“. In: *ACM Computing Surveys* 57.2 (2024), S. 1–36. DOI: 10.1145/3695462.
- [9] Olumese Anthony Abieba u. a. „Advanced Observability and Monitoring Practices for Enhancing Production Environment Reliability“. In: (2025). URL: <https://worldscientificnews.com/wp-content/uploads/2025/06/WSN-204-2025-270-296.pdf>.
- [10] Carlos Albuquerque und Filipe F Correia. „Tracing and Metrics Design Patterns for Monitoring Cloud-native Applications“. In: *arXiv preprint arXiv:2510.02991* (2025). URL: <https://arxiv.org/abs/2510.02991>.
- [11] A. Banerjee und R. Singh. „A Comparative Analysis of System Monitoring Architecture: Evaluating Prometheus, ELK Stack, and Custom Dashboards for Performance and Scalability“. In: *International Journal of Computer Applications* (2024). DOI: 10.21275/SR25917095133.

Hinweis zur KI-Nutzung: Bei der Erstellung dieses Exposés wurden KI-Tools (GitHub Copilot, ChatGPT, Consensus, Elicit) zur Unterstützung bei Recherche und Formulierung eingesetzt. Die inhaltliche Verantwortung liegt beim Autor.